

QUOTA: The Idea, Explained Simply

Anonymous, deposit-backed rate limits for AI agents, on Monad. Monad Metropolis · Track: Trust, Identity & AI Infrastructure

1. The one-minute version

Imagine a festival where every visitor must get through a gate, and the staff wants to stop anyone from cramming in 10,000 times.

- **Today's answer:** check everyone's ID and log who came in. That's safe for the festival but terrible for privacy: someone now holds a record of every visitor and everything they did.
- **QUOTA's answer:** you put down a **deposit** and receive a **wristband**. The gate can verify the wristband is real **without learning who you are**. You may enter, say, 100 times an hour. If you try a 101st time, the wristband **accidentally reveals your secret key**, anyone who sees it can claim your deposit, and you're out.

That's the whole idea. Replace "visitors" with **AI agents** and "festival gate" with **any API, tool server or website**.

Rate limiting that is private for the user, enforceable for the service, and run by nobody.

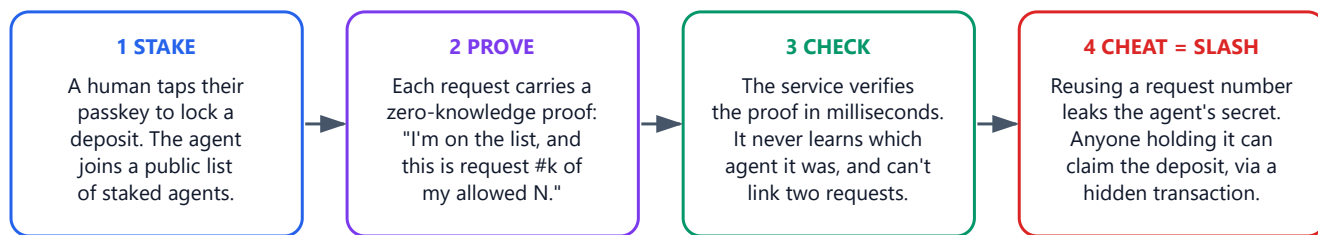
2. The problem, in plain words

AI agents now send a huge amount of internet traffic: research agents, shopping agents, coding agents calling tools. Anyone running a free or cheap service has to stop a single agent from hammering it. Every current fix has a catch:

Today's fix	The catch
API keys / accounts	The service sees and logs everything your agent does. Agents also can't fill in signup forms.
Block by IP address	Honest agents share cloud IPs and get blocked; abusers rent new IPs for pennies.
CAPTCHA	Built to stop bots. Your legitimate agent <i>is</i> a bot.
Pay per call	Every payment is public, so your agent's whole history is visible. Many services don't want to charge anyway.
Apple/Cloudflare "Privacy Pass" (ARC)	Private, but a company decides who is allowed in . One party controls the trust layer.

What's missing: a way to limit abuse that doesn't track users and doesn't need a gatekeeper. That is exactly the question this track asks: *trust infrastructure that no single platform can capture*.

3. How it works: four steps



Honest agents stay anonymous. Cheating costs real money. Nobody is in charge.

The clever bit (no maths needed). Each request includes one point on a secret line. One point reveals nothing. Two points on the same line reveal the line, and the line *is* the agent's secret key. An honest agent never sends two points for the same request number. A cheating agent has no choice. That is how breaking the rules automatically exposes the cheater.

4. Who's who

Role	Plain description
Operator	The human or team that owns agents. Locks the deposit and approves with a passkey (fingerprint / face / PIN on their device).
Agent	The AI program that makes requests. Holds a secret key.
Service	The API, MCP tool server or website being protected. Sets how many requests per hour, and which "trust lists" to accept.
Slasher	Automatic software on the service side. When it catches a repeat, it recovers the secret and claims the deposit.
The chain (Monad)	Holds deposits and the public list. Enforces the rules. No company in the middle.

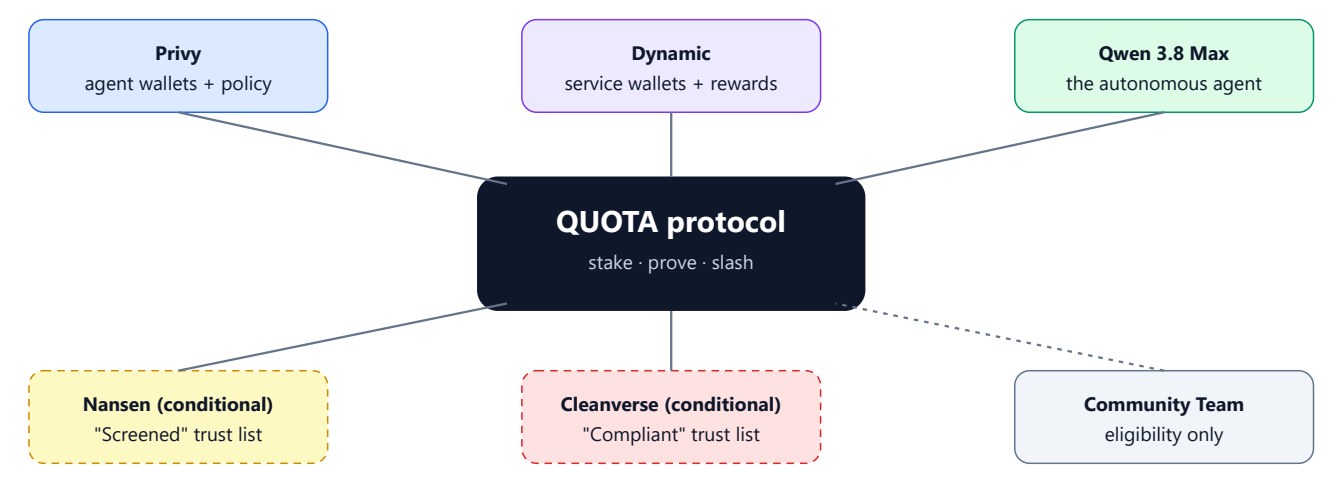
5. The three Monad building blocks, and why each is needed

Monad feature	Where QUOTA uses it	What breaks without it
Passkey check (P256 precompile)	Locking, unlocking and changing a deposit all need a real passkey tap, verified on-chain by Monad's built-in P256 support.	A stolen agent key could drain the deposit. The passkey keeps the human in charge of the money.
BTX (encrypted mempool)	The "claim the deposit" transaction contains the cheater's secret. If it were readable before it lands, someone could copy it and steal the reward. BTX keeps it sealed until its place is fixed.	Slash rewards get front-run. Fallback if BTX isn't available on testnet: commit a hidden fingerprint first, reveal later.

Monad feature	Where QUOTA uses it	What breaks without it
ERC-8004 (agent registry)	Optional: an agent can choose to link its quota identity to its public agent profile, so a slash becomes a visible reputation event.	Nothing core breaks. It adds accountability only for agents that opt in.

6. What each bounty does for QUOTA

The rule we used: **if we remove the sponsor's piece, does QUOTA break or get clearly worse?** If not, we didn't count it.



Privy: agent wallet infrastructure (Core)

In plain words: every agent needs a wallet, but there's no human sitting there approving each action. Privy provides a wallet the agent can use on its own, with safety rails the human sets.

- The agent's wallet is **owned by the human operator**. Our software gets limited "runtime" permission to use it.
- A **policy** says the agent's wallet may only talk to the QUOTA contract, with a spending cap. If the agent software is hacked, the attacker is boxed in.
- The agent's **secret key is derived from a wallet signature**, so it's recoverable with the wallet and never saved in plain text.
- We use Privy's signing-only mode, so our own code sends the transaction (needed for the BTX path).
- *Without it:* we'd have to hand-build agent key custody and policy limits. That is the hardest part of safely running autonomous agents.

Dynamic: service-side wallets (Core, riskiest)

In plain words: the service owner also needs a wallet, to receive rewards and to let the automatic slasher act without a prompt every time.

- The service owner logs in with Dynamic and gets an **embedded wallet** that receives slash rewards.
- **Delegated access** lets the backend slasher sign slash transactions from that wallet without a pop-up each time, and the owner can revoke it any time.
- This shows QUOTA works with **different wallet providers on each side** (agents on Privy, services on Dynamic), which is good developer experience.

- *Without it:* services need to manage a raw private key on a server. **Safety valve:** if Dynamic doesn't work on Monad in our first-day test, we drop it rather than fake it.

Qwen 3.8 Max: the autonomous agent (Core for the demo)

In plain words: QUOTA is built for agents, so we need a real one. "Scout" is a Qwen-powered agent that plans research across several protected services.

- Scout has tools: *check my remaining quota, call a tool on a server, top up my deposit, switch to another server.*
- It treats quota like a **budget**: plans its calls, backs off before the limit, decides when raising its deposit is worth it.
- A "misbehaving" mode deliberately cheats so the audience sees the slash happen.
- A published article on how Qwen was used is part of this bounty.
- *Honest note:* the protocol doesn't need an AI. Qwen is the realistic workload that proves the problem, not a core dependency.

Nansen: a "Screened" trust list (Conditional, first to be cut)

In plain words: a deposit stops most cheating, but a banned cheater could come back from a related wallet. Nansen's wallet intelligence can spot that.

- A **screener** checks a new operator wallet using Nansen data (labels, related wallets, funding source, past partners).
- If it looks clean, the screener signs a stamp, and the wallet can join a separate **Screened list**.
- Services choose which lists they accept. Anyone can run a screener, so Nansen doesn't become a new gatekeeper.
- *Without it:* the Screened list doesn't exist. Only built if time allows after the core is working.

Cleanverse: a "Compliant" trust list (Conditional, decided on day 1)

In plain words: some services (regulated data, finance) must only deal with verified parties.

- The deposit is held as a **Cleanverse verified asset**. It can only move between wallets with **verified identity** on-chain, including the slash payout.
- Requests stay anonymous; only the *operator* is verified.
- *Honest tension:* Cleanverse is a central issuer, which QUOTA's open list avoids. So this is strictly opt-in per service.
- *Blocker:* their docs need an invitation code, so a day-1 check decides whether we can build it at all.

Community Team: eligibility only

Not a feature. If our team's profile names an onboarded community group, the same project is also considered for this bounty. We need to know which group.

7. A story from start to finish

An MCP server offers a free web-search tool: **100 calls per hour per staked agent.**

1. **Alice** logs in, taps her passkey, and locks 10 MON for her research agent. The agent joins the public list. (*Privy wallet, P256 check*)
2. **Scout** (powered by Qwen) plans a task and makes 60 calls. The server sees 60 valid, unlinkable proofs. It doesn't know they came from one agent, or that it was Alice's.
3. **A scraper** staked the same way and fires 5,000 calls. On call 101 it must reuse a request number, which exposes its secret key.
4. The **slasher** (service side, Dynamic wallet) recovers the secret, sends the claim through a hidden BTX transaction, and receives part of the scraper's deposit. (*BTX, Dynamic*)
5. If the scraper stakes again, the same thing happens. Splitting into many small deposits doesn't help: allowance scales with deposit.

8. Limits we state honestly

- **Not proof of being human.** A passkey isn't a person. The *deposit* is what stops abuse.
- **Limits are per service.** Each service sets its own N.
- **The deposit has to be worth more than the abuse.** Services pick a minimum.
- **Proofs take time.** Not suited to ultra-fast public RPC; aimed at tool servers, APIs and scraping targets.
- **The operator is public** when they enroll. Anonymity is among everyone who is staked.
- **BTX might not be live on testnet.** If so, we ship the commit-then-reveal fallback and say so.
- **Not brand-new cryptography.** It builds on audited RLN circuits. What's new is the on-chain deposit, the passkey custody, and the sealed claim.

9. Why it fits the track

Track question	QUOTA's answer
Trust layer	Services trust that callers are <i>bounded</i> , without knowing <i>who</i> .
AI-native internet	Built for agent traffic, where old tools assume humans.
Privacy-preserving	Requests can't be linked to each other or to the owner.
Composable	One deposit works at every service that accepts the proof.
Impossible to capture	No issuer. The contract, not a company, enforces it. This is the key difference from Apple/Cloudflare's version.
A primitive, not an app	Deliverable is a contract, SDKs and a drop-in middleware.

The pitch in one line: "Apple and Cloudflare's private rate limits, without the company in the middle."